

Terraform in DevOps – Part 2

CSC-535 Cloud DevOps

Muhammad Tausif

Department of Computer Science
QAU

Lecture Roadmap

- 1 Recap of Part 1
- 2 Terraform Modules
- 3 Expressions, Functions, and Loops
- 4 Provisioners
- 5 Workspaces
- 6 Terraform Import and State Migration
- 7 Terraform with Kubernetes
- 8 Real-World AWS Project Walkthrough
- 9 Terraform Cloud and Team Workflows
- 10 CI/CD Integration Patterns
- 11 Security Best Practices
- 12 Testing Terraform Code
- 13 Terraform Anti-Patterns and Troubleshooting
- 14 Course Summary and Final Perspective

Quick Recap: What We Covered Last Time

In Terraform Part 1, we learned:

- Terraform architecture: Core, Providers, State.
- HCL syntax and data types.
- Provider configuration and versioning.
- Resource lifecycle (CRUD) and dependencies.
- Variables, outputs, and sensitive values.
- State management: local vs remote backends.
- Data sources for reading existing infrastructure.
- CLI commands: init, plan, apply, destroy, fmt, validate.

Today's Focus

Advanced patterns that make Terraform production-ready.

Why Modules?

Without modules:

- All resources in one directory.
- Code grows unmanageable.
- Copy-paste between projects.
- No standardization across teams.

With modules:

- Encapsulate related resources together.
- Reuse across multiple projects.
- Enforce organizational standards.
- Abstract complexity behind clean interfaces.

Analogy

Modules are to Terraform what **functions** are to programming.

What is a Module?

A module is simply a **directory containing .tf files**.

- Every Terraform project is already a module (the **root module**).
- A module called from another module is a **child module**.

Module hierarchy:

Root Module → Child Module A

Root Module → Child Module B

Key Idea

Modules are not a separate feature. They are how Terraform naturally organizes code.

Module Structure

A well-designed module contains:

File	Purpose
<code>main.tf</code>	Resource definitions
<code>variables.tf</code>	Input variables (the interface)
<code>outputs.tf</code>	Output values (return values)
<code>versions.tf</code>	Provider version constraints
<code>README.md</code>	Documentation for users

Design Principle

A module should do **one thing well**. A VPC module creates VPCs. An EC2 module creates instances.

Writing a Module: VPC Example

Directory: modules/vpc/

modules/vpc/variables.tf:

```
variable "vpc_cidr" {  
    description = "CIDR block for the VPC"  
    type        = string  
}  
  
variable "vpc_name" {  
    description = "Name tag for the VPC"  
    type        = string  
}
```

These variables are the module's **input interface**.

VPC Module: Main Resources

modules/vpc/main.tf:

```
resource "aws_vpc" "this" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true

  tags = {
    Name = var.vpc_name
  }
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id

  tags = {
    Name = "${var.vpc_name}-igw"
  }
}
```


VPC Module: Outputs

modules/vpc/outputs.tf:

```
output "vpc_id" {  
    description = "ID of the created VPC"  
    value       = aws_vpc.this.id  
}  
  
output "vpc_cidr" {  
    description = "CIDR block of the VPC"  
    value       = aws_vpc.this.cidr_block  
}  
  
output "igw_id" {  
    description = "Internet Gateway ID"  
    value       = aws_internet_gateway.this.id  
}
```

Outputs are the module's **return values**.

Calling the Module

In the root module (main.tf):

```
module "production_vpc" {  
    source      = "../modules/vpc"  
    vpc_cidr    = "10.0.0.0/16"  
    vpc_name    = "production"  
}
```

```
module "staging_vpc" {  
    source      = "../modules/vpc"  
    vpc_cidr    = "10.1.0.0/16"  
    vpc_name    = "staging"  
}
```

Reusability in Action

Same module code creates two completely separate VPCs with different configurations.

Accessing Module Outputs

Use module outputs in other resources:

```
resource "aws_subnet" "public" {
  vpc_id      = module.production_vpc.vpc_id
  cidr_block  = "10.0.1.0/24"

  tags = {
    Name = "public-subnet"
  }
}

output "prod_vpc_id" {
  value = module.production_vpc.vpc_id
}
```

Syntax

```
module.<MODULE_NAME>.<OUTPUT_NAME>
```

Module Sources

Modules can be loaded from multiple sources:

Source	Example
Local path	<code>./modules/vpc</code>
Terraform Registry	<code>hashicorp/consul/aws</code>
GitHub	<code>github.com/org/repo</code>
S3 bucket	<code>s3::https://bucket/vpc.zip</code>
Git repository	<code>git::https://repo.git</code>

Terraform Registry

`registry.terraform.io` hosts thousands of community and official modules.

Using Registry Modules

```
module "vpc" {  
    source      = "terraform-aws-modules/vpc/aws"  
    version    = "5.1.0"  
  
    name       = "production-vpc"  
    cidr       = "10.0.0.0/16"  
  
    azs        = ["us-east-1a", "us-east-1b"]  
    private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]  
    public_subnets  = ["10.0.101.0/24", "10.0.102.0/24"]  
  
    enable_nat_gateway = true  
}
```

Advantage

Community modules handle edge cases and best practices you might miss writing from scratch.

Module Versioning

Always pin module versions:

- `version = "5.1.0"` – exact version.
- `version = "~> 5.1"` – any 5.x patch version.
- `version = ">= 5.0, < 6.0"` – range constraint.

Why version pinning matters:

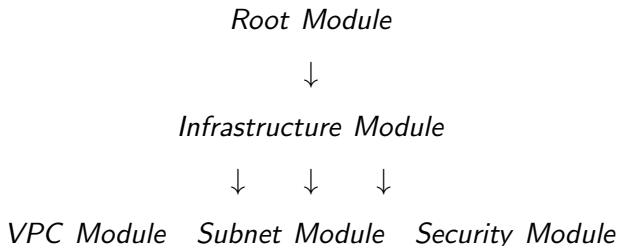
- Module updates can change behavior.
- Breaking changes can destroy resources.
- Reproducible builds require fixed versions.

Rule

Never use unpinned module versions in production.

Nested Modules

Modules can call other modules:



Real-World Pattern

Large organizations create a hierarchy: platform team publishes base modules, application teams compose them.

Module Best Practices

- 1 One module per logical component (VPC, database, app).
- 2 Define clear input variables with descriptions.
- 3 Expose only necessary outputs.
- 4 Write a `README.md` for every module.
- 5 Use semantic versioning when publishing.
- 6 Never hardcode values inside modules.
- 7 Test modules independently before composing.
- 8 Keep modules focused: 5–15 resources maximum.

Principle

A module should be understandable by reading its `variables.tf` and `outputs.tf` alone.

Why Expressions and Functions?

Real infrastructure is not static:

- Different environments need different sizes.
- Some resources are conditional.
- Lists of subnets must be generated dynamically.
- Tags must be computed from other values.

Terraform provides:

- Conditional expressions.
- Looping constructs (`count`, `for_each`).
- Built-in functions (`string`, `numeric`, `collection`).
- Dynamic blocks.

Conditional Expressions

Syntax: `condition ? true_value : false_value`

```
variable "environment" {  
    type      = string  
    default   = "dev"  
}  
  
resource "aws_instance" "web" {  
    instance_type = (  
        var.environment == "production"  
        ? "t3.large"  
        : "t2.micro"  
    )  
    # ...  
}
```

Use Case

Production gets bigger instances. Dev/staging get smaller ones. Same code handles both.

Conditional Resource Creation

Create a resource only when a condition is met:

```
variable "create_monitoring" {  
    type      = bool  
    default   = false  
}  
  
resource "aws_cloudwatch_metric_alarm" "cpu" {  
    count = var.create_monitoring ? 1 : 0  
  
    alarm_name    = "high-cpu"  
    metric_name   = "CPUUtilization"  
    threshold     = 80  
    # ...  
}
```

- `count = 1`: resource is created.
- `count = 0`: resource is skipped entirely.

count: Creating Multiple Resources

```
variable "instance_count" {
  type      = number
  default   = 3
}

resource "aws_instance" "web" {
  count          = var.instance_count
  ami            = "ami-0c55b159cbfaffe1f0"
  instance_type = "t2.micro"

  tags = {
    Name = "web-server-${count.index}"
  }
}
```

- Creates 3 instances: web-server-0, web-server-1, web-server-2.
- `count.index` gives the current iteration number.

Problems with count

`count` uses index-based identification:

- Resources are `aws_instance.web[0]`, `[1]`, `[2]`.
- If you remove item at index 1:
 - Index 2 becomes index 1.
 - Terraform **destroys and recreates** the shifted resource.

When to Use count

Only when resources are truly identical and order does not matter.

For named or unique resources, use `for_each` instead.

for_each with Map

```
variable "instances" {
  type = map(string)
  default = {
    web      = "t2.micro"
    api      = "t2.small"
    worker   = "t3.medium"
  }
}

resource "aws_instance" "server" {
  for_each      = var.instances
  ami           = "ami-0c55b159cbfaffe1f0"
  instance_type = each.value

  tags = {
    Name = each.key
  }
}
```

for_each vs count

Feature	count	for_each
Identifier	Numeric index	Map key or set value
Removal	Shifts all indexes	Only removes targeted item
Access	<code>count.index</code>	<code>each.key</code> , <code>each.value</code>
Best for	Identical copies	Named/unique resources

Industry Standard

Prefer `for_each` over `count` in production. It is safer and more predictable.

for_each with Set

```
variable "subnet_cidrs" {  
    type      = set(string)  
    default   = ["10.0.1.0/24", "10.0.2.0/24",  
                "10.0.3.0/24"]  
}  
  
resource "aws_subnet" "private" {  
    for_each    = var.subnet_cidrs  
    vpc_id      = aws_vpc.main.id  
    cidr_block  = each.value  
  
    tags = {  
        Name = "private-${each.value}"  
    }  
}
```

Creates one subnet per CIDR block. Removing a CIDR only destroys that specific subnet.

for Expressions

Transform collections inline:

```
# List transformation
locals {
  upper_names = [
    for name in var.names : upper(name)
  ]
}

# Map transformation
locals {
  instance_ids = {
    for k, v in aws_instance.server :
    k => v.id
  }
}
```

Think of it as

List comprehension in Python: `[f(x) for x in list]`

for Expression with Filter

Add conditions to filter elements:

```
variable "users" {  
  type = map(object({  
    role      = string  
    enabled   = bool  
  }))  
}  
  
locals {  
  active_admins = {  
    for name, user in var.users :  
    name => user  
    if user.role == "admin" && user.enabled  
  }  
}
```

Result

Only active admin users are included in the output map.

Dynamic Blocks

Generate repeated nested blocks dynamically:

```
variable "ingress_rules" {
  type = list(object({
    port          = number
    description    = string
  }))
  default = [
    { port = 80,   description = "HTTP" },
    { port = 443,  description = "HTTPS" },
    { port = 22,   description = "SSH" }
  ]
}
```

Dynamic Blocks: Usage

```
resource "aws_security_group" "web" {  
  name      = "web-sg"  
  vpc_id    = aws_vpc.main.id  
  
  dynamic "ingress" {  
    for_each = var.ingress_rules  
    content {  
      from_port      = ingress.value.port  
      to_port        = ingress.value.port  
      protocol        = "tcp"  
      cidr_blocks     = ["0.0.0.0/0"]  
      description     = ingress.value.description  
    }  
  }  
}
```

Benefit

No need to repeat ingress {} blocks manually. Data drives the

Built-in Functions Overview

Terraform provides 100+ built-in functions:

Category	Examples
String	upper, lower, format, join, split, replace, trim
Numeric	min, max, ceil, floor, abs
Collection	length, merge, concat, flatten, keys, values, lookup
Encoding	base64encode, jsonencode, yamldecode
Filesystem	file, templatefile, fileexists
IP/Network	cidrsubnet, cidrhost
Date/Time	timestamp, formatdate

Commonly Used Functions: Examples

```
# String
name = lower("PRODUCTION") # "production"
msg  = format("Hello, %s!", var.name)

# Collection
all_tags = merge(var.default_tags, {
  Name = "my-resource"
})
first_az = element(var.azs, 0)

# Networking
subnet = cidrsubnet("10.0.0.0/16", 8, 1)
# Result: "10.0.1.0/24"

# File
config = file("${path.module}/config.json")
```

templatefile Function

Render templates with variables:

userdata.tpl:

```
#!/bin/bash
echo "Server: ${server_name}"
echo "Environment: ${env}"
apt-get update && apt-get install -y ${packages}
```

main.tf:

```
resource "aws_instance" "web" {
  user_data = templatefile("userdata.tpl", {
    server_name = "web-01"
    env         = "production"
    packages    = "nginx curl"
  })
  # ...
}
```

Locals: Computed Values

Use locals to define computed intermediate values:

```
locals {  
    env_prefix = "${var.project}-${var.environment}"  
  
    common_tags = {  
        Project      = var.project  
        Environment  = var.environment  
        ManagedBy    = "Terraform"  
        Team         = var.team  
    }  
  
    is_production = var.environment == "production"  
}
```

- Avoid repeating complex expressions.
- Give meaningful names to computed values.
- Reference as `local.env_prefix`, `local.common_tags`.

Using Locals Throughout Configuration

```
resource "aws_instance" "web" {  
  instance_type = (  
    local.is_production ? "t3.large" : "t2.micro"  
  )  
  
  tags = merge(local.common_tags, {  
    Name = "${local.env_prefix}-web"  
    Role = "webserver"  
  })  
  # ...  
}
```

DRY Principle

Define once in locals, use everywhere. Changes propagate automatically.

What are Provisioners?

Provisioners run **scripts or commands** on resources after creation.

- Execute shell commands on a VM after it boots.
- Copy files to a remote machine.
- Run configuration management tools.

Three types:

- 1 `local-exec` – runs on the machine running Terraform.
- 2 `remote-exec` – runs on the created resource via SSH/WinRM.
- 3 `file` – copies files to the resource.

Runs a command on **your local machine**:

```
resource "aws_instance" "web" {  
  ami          = "ami-0c55b159cbfaffe1f0"  
  instance_type = "t2.micro"  
  
  provisioner "local-exec" {  
    command = "echo ${self.public_ip} >> hosts.txt"  
  }  
}
```

- Appends the new instance IP to a local file.
- `self` refers to the parent resource.

remote-exec Provisioner

Runs commands **on the created instance**:

```
resource "aws_instance" "web" {  
  # ...  
  
  connection {  
    type      = "ssh"  
    user      = "ubuntu"  
    private_key = file("~/.ssh/id_rsa")  
    host      = self.public_ip  
  }  
  
  provisioner "remote-exec" {  
    inline = [  
      "sudo apt-get update",  
      "sudo apt-get install -y nginx",  
      "sudo systemctl start nginx"  
    ]  
  }  
}
```

file Provisioner

Copies files from local machine to the resource:

```
resource "aws_instance" "web" {  
  # ...  
  
  connection {  
    type      = "ssh"  
    user      = "ubuntu"  
    private_key = file("~/.ssh/id_rsa")  
    host      = self.public_ip  
  }  
  
  provisioner "file" {  
    source      = "config/nginx.conf"  
    destination = "/tmp/nginx.conf"  
  }  
}
```

Provisioner Behavior Options

- **Creation-time** (default): runs only when resource is created.
- **Destroy-time**: runs before resource is destroyed.

On failure behavior:

- `on_failure = continue` – ignore errors, proceed.
- `on_failure = fail` – mark resource as tainted (default).

Important Warning

HashiCorp considers provisioners a **last resort**. Prefer cloud-init, user_data, or configuration management tools (Ansible) for server setup.

Why Provisioners are a Last Resort

Problems with provisioners:

- Provisioner failures leave resources in unknown state.
- Not tracked in Terraform state.
- Cannot detect drift in provisioned configuration.
- Break the declarative model.
- Make `terraform plan` less accurate.

	Instead of	Use
Better alternatives:	remote-exec for install	<code>user_data</code> / cloud-init
	file provisioner	Packer pre-built images
	Complex setup scripts	Ansible after Terraform

What are Workspaces?

Workspaces allow **multiple state files** for the same configuration.

- Each workspace has its own `terraform.tfstate`.
- Same code can manage dev, staging, and production.
- Isolated state per environment.

Default workspace: `default` (always exists).

Use Case

One Terraform project that manages multiple environments without code duplication.

Workspace Commands

List all workspaces

```
terraform workspace list
```

Create a new workspace

```
terraform workspace new staging
```

Switch to a workspace

```
terraform workspace select production
```

Show current workspace

```
terraform workspace show
```

Delete a workspace

```
terraform workspace delete staging
```

Using Workspace Name in Configuration

```
resource "aws_instance" "web" {
  instance_type = (
    terraform.workspace == "production"
    ? "t3.large"
    : "t2.micro"
  )

  tags = {
    Name = "web-${terraform.workspace}"
    Env  = terraform.workspace
  }
}
```

- `terraform.workspace` returns the current workspace name.
- Configuration adapts automatically per environment.

Workspace with Variable Maps

More structured approach:

```
variable "config" {
  type = map(object({
    instance_type = string
    min_size      = number
    max_size      = number
  }))
  default = {
    dev      = { instance_type = "t2.micro",
                  min_size     = 1, max_size = 1 }
    staging  = { instance_type = "t2.small",
                  min_size     = 1, max_size = 2 }
    production = { instance_type = "t3.large",
                    min_size     = 2, max_size = 10 }
  }
}

locals {
  env = var.config[terraform.workspace]
```

Workspaces: Limitations and Alternatives

Workspace limitations:

- All environments share the same code and backend.
- No access control between workspaces.
- State isolation is shallow (same bucket, different keys).
- Accidental workspace switch can be dangerous.

Alternative: **Directory-based separation:**

- `environments/dev/` – separate Terraform project.
- `environments/staging/` – separate Terraform project.
- `environments/prod/` – separate Terraform project.
- Shared modules in `modules/`.

Industry Preference

Many teams prefer directory separation for production environments and workspaces for non-critical ones.

Importing Existing Resources

Not all infrastructure starts with Terraform.

Common scenario:

- Team manually created AWS resources via console.
- Now wants to manage them with Terraform.
- Cannot destroy and recreate (production systems).

Solution: `terraform import`

- Brings existing resources under Terraform management.
- Adds the resource to the state file.
- Does **not** generate configuration code automatically.

Import Workflow

Step 1: Write the resource block (matching existing resource):

```
resource "aws_instance" "legacy_server" {  
    # Will be filled after import  
}
```

Step 2: Run import command:

```
terraform import aws_instance.legacy_server \  
    i-0abcdef1234567890
```

Step 3: Run terraform plan and fill in attributes until plan shows no changes.

Tedious but Necessary

Import requires manually writing the configuration to match the existing resource.

Import Block (Terraform 1.5+)

Newer Terraform supports declarative import:

```
import {  
  to = aws_instance.legacy_server  
  id = "i-0abcdef1234567890"  
}  
  
resource "aws_instance" "legacy_server" {  
  ami           = "ami-0c55b159cbfaffe1f0"  
  instance_type = "t2.micro"  
}
```

- Import becomes part of the configuration.
- Can be reviewed in terraform plan.
- More auditable than CLI import.

State Migration Scenarios

Common state operations in real projects:

Scenario	Command
Rename resource	<code>terraform state mv</code>
Remove from state	<code>terraform state rm</code>
Move to different state	<code>terraform state mv</code> <code>-state-out</code>
Migrate backend	<code>terraform init</code> <code>-migrate-state</code>
Replace provider	<code>terraform state</code> <code>replace-provider</code>

Golden Rule

Always backup your state file before running state manipulation commands.

Terraform and Kubernetes: Two Roles

Terraform interacts with Kubernetes at two levels:

- ❶ **Provision the cluster** (infrastructure layer):
 - Create EKS (AWS), AKS (Azure), or GKE (GCP).
 - Set up networking, IAM, node groups.
- ❷ **Deploy to the cluster** (application layer):
 - Create namespaces, deployments, services.
 - Manage ConfigMaps, secrets, ingress rules.

Best Practice

Use Terraform for cluster creation. Use Kubernetes manifests or Helm for application deployment.

Creating an EKS Cluster with Terraform

```
module "eks" {  
  source      = "terraform-aws-modules/eks/aws"  
  version    = "19.0"  
  
  cluster_name      = "production-cluster"  
  cluster_version   = "1.28"  
  vpc_id            = module.vpc.vpc_id  
  subnet_ids       = module.vpc.private_subnets  
  
  eks_managed_node_groups = {  
    general = {  
      desired_size = 2  
      min_size     = 1  
      max_size     = 5  
      instance_types = ["t3.medium"]  
    }  
  }  
}
```

Result

Terraform creates a fully configured Kubernetes cluster with managed node groups.

Kubernetes Provider in Terraform

```
provider "kubernetes" {  
  host = module.eks.cluster_endpoint  
  cluster_ca_certificate = base64decode(  
    module.eks.cluster_certificate_authority_data  
  )  
  exec {  
    api_version = "client.authentication.k8s.io/v1"  
    command     = "aws"  
    args = ["eks", "get-token",  
           "--cluster-name",  
           module.eks.cluster_name]  
  }  
}
```

Connection

Terraform authenticates to the EKS cluster it just created and can manage Kubernetes resources.

Deploying Kubernetes Resources via Terraform

```
resource "kubernetes_namespace" "app" {
  metadata {
    name = "my-application"
  }
}

resource "kubernetes_deployment" "nginx" {
  metadata {
    name          = "nginx"
    namespace     = kubernetes_namespace.app.
                      metadata[0].name
  }
  spec {
    replicas = 3
    selector {
      match_labels = { app = "nginx" }
    }
    # ... template spec
  }
}
```

Helm Provider in Terraform

Deploy Helm charts via Terraform:

```
provider "helm" {  
  kubernetes {  
    host = module.eks.cluster_endpoint  
    cluster_ca_certificate = base64decode(  
      module.eks.cluster_certificate_authority_data  
    )  
    # ... authentication  
  }  
}  
  
resource "helm_release" "ingress_nginx" {  
  name          = "ingress-nginx"  
  repository    = "https://kubernetes.github.io/  
    ingress-nginx"  
  chart         = "ingress-nginx"  
  namespace    = "ingress"  
}
```

Terraform + Kubernetes: Complete Picture

Full infrastructure-to-deployment workflow:

- 1 Terraform creates VPC, subnets, security groups.
- 2 Terraform creates EKS cluster and node groups.
- 3 Terraform creates ECR (container registry).
- 4 Docker builds and pushes application image to ECR.
- 5 Terraform (or Helm/kubectl) deploys app to EKS.
- 6 Terraform creates Route53 DNS records.
- 7 Terraform creates CloudWatch monitoring.

Network → Cluster → Registry → Image → Deploy → DNS → Monitor

Project: Three-Tier Web Application on AWS

We will build a complete infrastructure:

- **Networking:** VPC, public/private subnets, NAT Gateway.
- **Compute:** EC2 instances in an Auto Scaling Group.
- **Load Balancing:** Application Load Balancer (ALB).
- **Database:** RDS MySQL in private subnet.
- **Security:** Security groups for each tier.

Architecture

Internet → ALB → Web Servers → Database

Project File Structure

File	Content
providers.tf	AWS provider config
variables.tf	All input variables
vpc.tf	VPC, subnets, IGW, NAT
security.tf	Security groups
alb.tf	Load balancer and target group
asg.tf	Launch template and ASG
rds.tf	Database instance
outputs.tf	ALB DNS, DB endpoint
terraform.tfvars	Environment values

Project: Provider and Variables

```
providers.tf:

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
  required_version = ">= 1.5.0"
}

provider "aws" {
  region = var.aws_region
}
```

Project: Core Variables

variables.tf:

```
variable "aws_region" {  
    description = "AWS region"  
    type        = string  
    default     = "us-east-1"  
}
```

```
variable "project_name" {  
    description = "Project identifier"  
    type        = string  
    default     = "webapp"  
}
```

```
variable "environment" {  
    description = "Deployment environment"  
    type        = string  
    default     = "production"  
}
```

Project: VPC and Subnets

vpc.tf:

```
resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = { Name = "${var.project_name}-vpc" }
}

resource "aws_subnet" "public" {
  count          = 2
  vpc_id        = aws_vpc.main.id
  cidr_block     = cidrsubnet(var.vpc_cidr, 8, count.index)
  availability_zone = data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true
  tags = { Name = "${var.project_name}-public-${count.index}" }
}

resource "aws_subnet" "private" {
  count          = 2
  vpc_id        = aws_vpc.main.id
  cidr_block     = cidrsubnet(var.vpc_cidr, 8, count.index + 10)
  availability_zone = data.aws_availability_zones.available.names[count.index]
  tags = { Name = "${var.project_name}-private-${count.index}" }
}
```

Project: Internet and NAT Gateways

```
resource "aws_internet_gateway" "main" {  
    vpc_id = aws_vpc.main.id  
    tags   = { Name = "${var.project_name}-igw" }  
}
```

```
resource "aws_eip" "nat" {  
    domain = "vpc"  
}
```

```
resource "aws_nat_gateway" "main" {  
    allocation_id = aws_eip.nat.id  
    subnet_id     = aws_subnet.public[0].id  
    tags = { Name = "${var.project_name}-nat" }  
}
```

- Internet Gateway: allows public subnets to reach internet.
- NAT Gateway: allows private subnets to reach internet (outbound only).

Project: Security Groups

security.tf:

```
resource "aws_security_group" "alb" {
  name      = "${var.project_name}-alb-sg"
  vpc_id    = aws_vpc.main.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

Project: Web Server Security Group

```
resource "aws_security_group" "web" {
  name      = "${var.project_name}-web-sg"
  vpc_id    = aws_vpc.main.id

  ingress {
    from_port      = 80
    to_port        = 80
    protocol       = "tcp"
    security_groups = [
      aws_security_group.alb.id
    ]
  }

  egress {
    from_port      = 0
    to_port        = 0
    protocol       = "-1"
    cidr_blocks    = ["0.0.0.0/0"]
  }
}
```

Project: Application Load Balancer

alb.tf:

```
resource "aws_lb" "main" {  
  name           = "${var.project_name}-alb"  
  internal       = false  
  load_balancer_type = "application"  
  security_groups = [aws_security_group.alb.id]  
  subnets       = aws_subnet.public[*].id  
}
```

```
resource "aws_lb_target_group" "web" {  
  name      = "${var.project_name}-tg"  
  port      = 80  
  protocol  = "HTTP"  
  vpc_id    = aws_vpc.main.id
```

```
  health_check {  
    path           = "/"  
    healthy_threshold = 2  
    unhealthy_threshold = 5
```

Project: ALB Listener

```
resource "aws_lb_listener" "http" {  
  load_balancer_arn = aws_lb.main.arn  
  port              = 80  
  protocol          = "HTTP"  
  
  default_action {  
    type              = "forward"  
    target_group_arn = aws_lb_target_group.web.arn  
  }  
}
```

Traffic flow:

User → ALB : 80 → Target Group → EC2 Instances

Project: Launch Template and Auto Scaling

asg.tf:

```
resource "aws_launch_template" "web" {
  name_prefix    = "${var.project_name}-lt-"
  image_id       = data.aws_ami.amazon_linux.id
  instance_type  = "t2.micro"

  vpc_security_group_ids = [aws_security_group.web.id]

  user_data = base64encode(<<-EOF
    #!/bin/bash
    yum update -y
    yum install -y httpd
    systemctl start httpd
    echo "<h1>${var.project_name} - $(hostname)</h1>" > /var/www/html/index.html
  EOF
)
}

resource "aws_autoscaling_group" "web" {
  desired_capacity = 2
  max_size         = 4
  min_size         = 1
  vpc_zone_identifier = aws_subnet.private[*].id
  target_group_arns  = [aws_lb_target_group.web.arn]

  launch_template {
    id      = aws_launch_template.web.id
    version = "$Latest"
  }
}
```

Project: RDS Database

rds.tf:

```
resource "aws_db_subnet_group" "main" {
  name          = "${var.project_name}-db-subnet"
  subnet_ids    = aws_subnet.private[*].id
}

resource "aws_db_instance" "main" {
  identifier          = "${var.project_name}-db"
  engine              = "mysql"
  engine_version      = "8.0"
  instance_class      = "db.t3.micro"
  allocated_storage   = 20
  db_name             = "appdb"
  username            = var.db_username
  password            = var.db_password
  db_subnet_group_name = aws_db_subnet_group.main.name
  vpc_security_group_ids = [
    aws_security_group.db.id
  ]
}
```

Project: Outputs

outputs.tf:

```
output "alb_dns_name" {
    description = "ALB public DNS"
    value       = aws_lb.main.dns_name
}

output "db_endpoint" {
    description = "RDS connection endpoint"
    value       = aws_db_instance.main.endpoint
}

output "vpc_id" {
    description = "VPC ID"
    value       = aws_vpc.main.id
}
```

After Apply

Access the application at: http://<alb_dns_name>

Project Architecture Summary

Resources created by this project:

Layer	Resources
Networking	VPC, 2 public + 2 private subnets
Gateways	Internet Gateway, NAT Gateway
Security	3 security groups (ALB, Web, DB)
Compute	Launch template, ASG (2–4 instances)
Load Balancing	ALB, target group, listener
Database	RDS MySQL in private subnet

Total: **15+** resources from approximately **200 lines** of Terraform code.

What is Terraform Cloud?

Terraform Cloud is HashiCorp's managed platform:

- Remote state storage with encryption.
- State locking automatically.
- Remote plan and apply execution.
- Team access controls and governance.
- Private module registry.
- Policy enforcement (Sentinel).

Free Tier

Terraform Cloud offers a free tier for up to 5 users. Suitable for small teams and learning.

Connecting to Terraform Cloud

```
terraform {  
  cloud {  
    organization = "my-organization"  
  
    workspaces {  
      name = "webapp-production"  
    }  
  }  
}
```

Workflow changes:

- `terraform init` connects to Terraform Cloud.
- `terraform plan` runs remotely.
- `terraform apply` runs remotely with approval.
- State is stored and encrypted in the cloud.

Team Workflow with Terraform Cloud

Typical enterprise workflow:

- 1 Developer creates a branch and modifies `.tf` files.
- 2 Pull request triggers speculative `terraform plan`.
- 3 Team reviews plan output in the PR.
- 4 After merge, Terraform Cloud runs `apply`.
- 5 Approval required from designated reviewers.
- 6 Applied changes are logged and auditable.

GitOps for Infrastructure

Git becomes the single source of truth. All changes go through code review.

Terraform Cloud vs Self-Managed

Feature	Self-Managed	Terraform Cloud
State storage	S3/GCS bucket	Built-in
Locking	DynamoDB/manual	Automatic
Remote execution	CI/CD scripts	Built-in
Access control	IAM policies	RBAC
Cost	Infrastructure cost	Free / paid tiers
Setup effort	High	Low

Decision Factor

Small teams: Terraform Cloud. Large enterprises: often self-managed for compliance.

Terraform in CI/CD Pipelines

Terraform integrates with all major CI/CD tools:

- GitHub Actions
- GitLab CI/CD
- Jenkins
- Azure DevOps
- CircleCI

Standard pipeline stages:

- 1 Checkout code.
- 2 `terraform init`
- 3 `terraform fmt -check`
- 4 `terraform validate`
- 5 `terraform plan`
- 6 Manual approval gate.
- 7 `terraform apply`

GitHub Actions Example

```
name: Terraform
on:
  push:
    branches: [main]
  pull_request:

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3

      - name: Terraform Init
        run: terraform init

      - name: Terraform Format Check
        run: terraform fmt -check

      - name: Terraform Validate
        run: terraform validate

      - name: Terraform Plan
        run: terraform plan -no-color
        if: github.event_name == 'pull_request'

      - name: Terraform Apply
        run: terraform apply -auto-approve
        if: github.ref == 'refs/heads/main'
```

Jenkins Pipeline Example

```
pipeline {
  agent any

  stages {
    stage('Init') {
      steps { sh 'terraform init' }
    }
    stage('Validate') {
      steps { sh 'terraform validate' }
    }
    stage('Plan') {
      steps { sh 'terraform plan -out=tfplan' }
    }
    stage('Approve') {
      steps {
        input message: 'Apply infrastructure changes?'
      }
    }
    stage('Apply') {
      steps { sh 'terraform apply tfplan' }
    }
  }
}
```

Key Feature

The input step pauses the pipeline for human approval before applying changes.

CI/CD Best Practices for Terraform

- 1 Always run `fmt` and `validate` in CI.
- 2 Save plan output and apply the **exact** plan.
- 3 Require manual approval for production applies.
- 4 Use separate pipelines per environment.
- 5 Store Terraform state in a remote backend.
- 6 Use service accounts with least-privilege permissions.
- 7 Post plan output as PR comments for review.
- 8 Never use `-auto-approve` for production.

Principle

Infrastructure changes should be as rigorously reviewed as application code changes.

Terraform Security Checklist

- 1 **State encryption:** always encrypt state at rest.
- 2 **No secrets in code:** use variables, vault, or environment variables.
- 3 **Least privilege:** Terraform service accounts should have minimal IAM permissions.
- 4 **State access control:** restrict who can read/write state.
- 5 **Audit logging:** enable CloudTrail/audit logs for API calls.
- 6 **Pin versions:** lock provider and module versions.

Managing Secrets with Terraform

Methods for handling secrets:

Method	How	Security
Environment vars	TF_VAR_password	Medium
tfvars file	password = "..."	Low
AWS Secrets Manager	data source lookup	High
HashiCorp Vault	Provider integration	High
SOPS	Encrypted files in Git	High

Never Do This

Never commit passwords, API keys, or tokens in `.tf` or `.tfvars` files to Git.

Secrets Manager Data Source

Read secrets at apply time:

```
data "aws_secretsmanager_secret_version" "db" {
  secret_id = "production/database"
}

locals {
  db_creds = jsondecode(
    data.aws_secretsmanager_secret_version.db.
      secret_string
  )
}

resource "aws_db_instance" "main" {
  username = local.db_creds.username
  password = local.db_creds.password
  # ...
}
```

Policy as Code: Sentinel

Sentinel enforces governance rules:

- Prevent creating resources without tags.
- Restrict instance types to approved list.
- Require encryption on all storage resources.
- Block public access to databases.
- Enforce naming conventions.

How It Works

Policies run between `plan` and `apply`. If a policy fails, the `apply` is blocked.

Available in Terraform Cloud (Team & Governance tier).

Why Test Infrastructure Code?

Terraform code can:

- Create wrong-sized resources (costly).
- Open security groups too broadly.
- Misconfigure networking (broken connectivity).
- Accidentally destroy production data.

Testing levels:

- 1 **Static analysis:** terraform validate, tflint.
- 2 **Plan inspection:** review plan output programmatically.
- 3 **Integration testing:** apply in sandbox and verify.
- 4 **Compliance testing:** policy checks with Sentinel/OPA.

tflint: Static Analysis

tflint catches errors that validate misses:

```
# Install tflint
brew install tflint

# Run tflint
tflint --init
tflint
```

What tflint detects:

- Invalid instance types (e.g., t2.superxlarge).
- Deprecated resource arguments.
- AWS-specific misconfigurations.
- Naming convention violations.

CI Integration

Run tflint alongside terraform validate in every pipeline.

Terraform Test Framework (1.6+)

Terraform 1.6 introduced native testing:

tests/vpc.tftest.hcl:

```
run "vpc_creation" {
  command = plan

  assert {
    condition = (
      aws_vpc.main.cidr_block == "10.0.0.0/16"
    )
    error_message = "VPC CIDR mismatch"
  }

  assert {
    condition = (
      aws_vpc.main.enable_dns_support == true
    )
    error_message = "DNS support must be enabled"
  }
}
```

Testing Best Practices

- 1 Run `terraform validate` on every commit.
- 2 Run `tflint` on every commit.
- 3 Write `terraform test` assertions for critical resources.
- 4 Use a sandbox AWS account for integration tests.
- 5 Always destroy test infrastructure after tests complete.
- 6 Test modules independently before composing.
- 7 Automate all testing in CI pipeline.

Cost Warning

Integration tests create real cloud resources. Always clean up and use budget alerts.

Common Terraform Anti-Patterns

- ❶ **Mega-monolith:** all resources in one state file.
- ❷ **No modules:** everything in root module with copy-paste.
- ❸ **Hardcoded values:** no variables, impossible to reuse.
- ❹ **Local state in production:** no locking, no backup.
- ❺ **Unpinned versions:** providers/modules update unexpectedly.
- ❻ **Manual state edits:** editing `.tfstate` JSON directly.
- ❼ **Over-provisioning:** applying blindly without reviewing plan.

Rule of Thumb

If the plan output is too long to review, your state is too large. Split into smaller states.

Troubleshooting Common Errors

Error	Solution
Provider not found	Run <code>terraform init</code>
State locked	Wait or force-unlock
Resource already exists	Import it or remove manually
Dependency cycle	Refactor resource references
Permission denied	Check IAM/cloud credentials
Timeout during apply	Check network, increase time-out
State version mismatch	Upgrade Terraform version

Debugging

Set `TF_LOG=DEBUG` for verbose output. Check `crash.log` for internal errors.

Debugging Terraform

```
# Enable debug logging
export TF_LOG=DEBUG
export TF_LOG_PATH=terraform-debug.log
```

```
# Run with debug
terraform plan
```

```
# Disable debug
unset TF_LOG
```

Log levels (from most to least verbose):

- TRACE – extremely detailed.
- DEBUG – detailed internal operations.
- INFO – general progress information.
- WARN – warnings.
- ERROR – errors only.

Complete DevOps Toolchain We Have Covered

Across all lectures in this course:

Tool	Layer	Purpose
Docker	Packaging	Containerize applications
Kubernetes	Orchestration	Deploy at scale
Terraform	Infrastructure	Create cloud resources
Ansible	Configuration	Configure servers
Jenkins	Pipeline	Automate CI/CD
Git	Versioning	Source of truth

Production Pipeline

Git → Jenkins → Terraform → Ansible → Docker → Kubernetes

Terraform Part 2 Summary

Today we covered:

- Modules: structure, sources, registry, versioning, nesting.
- Expressions: conditionals, count, for_each, for, dynamic blocks.
- Built-in functions and locals.
- Provisioners: local-exec, remote-exec, file.
- Workspaces: multi-environment management.
- Import: bringing existing resources under Terraform.
- Kubernetes integration: EKS, providers, Helm.
- Real-world AWS project: complete three-tier architecture.
- Terraform Cloud and team workflows.
- CI/CD integration: GitHub Actions, Jenkins pipelines.
- Security: secrets management, policy as code.
- Testing: validate, tflint, terraform test.
- Anti-patterns and troubleshooting.

Key Takeaways from Terraform Lectures

- 1 Infrastructure is code – treat it with the same discipline as application code.
- 2 Modules are the building blocks of scalable infrastructure.
- 3 State management is critical – always use remote state with locking.
- 4 Test your infrastructure before deploying to production.
- 5 Security is not optional – encrypt state, manage secrets properly.
- 6 CI/CD for infrastructure follows the same patterns as application delivery.

Final Thought

Modern DevOps = Code + Automation + Culture

Terraform is the tool that turns infrastructure into code.

Recommended Next Steps for Students

- 1 Install Terraform and complete the local file exercise.
- 2 Create a free AWS account and deploy the three-tier project.
- 3 Write your own VPC module with inputs and outputs.
- 4 Set up a GitHub Actions pipeline with Terraform.
- 5 Explore the Terraform Registry for community modules.
- 6 Practice importing existing cloud resources.

Resources:

- Terraform Documentation: developer.hashicorp.com/terraform
- Terraform Registry: registry.terraform.io
- HashiCorp Learn: developer.hashicorp.com/tutorials